

LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL KE 2

Linked List dalam Python



Disusun Oleh:

Nama : Noufal Aji Prasetyo

NPM : 2340506059

Kelas : Rombel 3

Program Studi S1 Teknologi Informasi

Fakultas Teknik, Universitas Tidar

Genap 2024/2025

A. Tujuan Praktikum

Agar setiap mahasiswa tahu cara mengaplikasikan struktur data linked list dalam python. Dan mahasiswa mampu memahami algoritma struktur data linked list, macam macam model linked list, dan fungsi pada setiap linked list

B. Dasar Teori

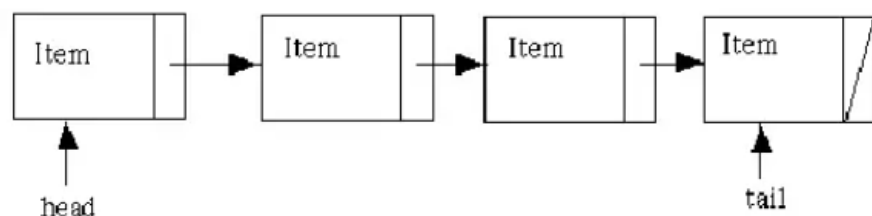
1. Linked list

Linked list atau dikenal juga dengan sebutan senarai berantai adalah struktur data yang terdiri dari urutan record data dimana setiap record memiliki field yang menyimpan alamat/referensi dari record selanjutnya (dalam urutan). Elemen data yang dihubungkan dengan link pada linked list disebut Node. Biasanya didalam suatu linked list, terdapat istilah head dan tail. Head adalah elemen yang berada pada posisi pertama dalam suatu linked list. Tail adalah elemen yang berada pada posisi terakhir dalam suatu linked list

2. Macam-Macam Linked List

a. Single Linked List

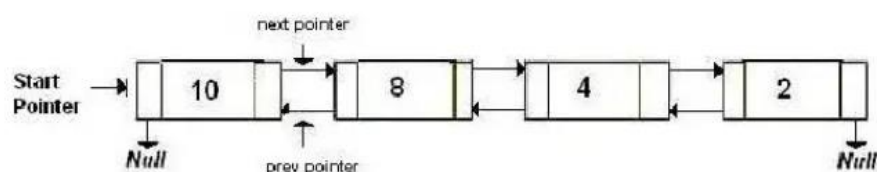
Single Linked List merupakan suatu linked list yang hanya memiliki satu variable pointer saja. Dimana pointer tersebut menunjukan ke node selanjutnya. Biasanya field pada tail menunjukan ke Null



Gambar 1.1

b. Double Linked List

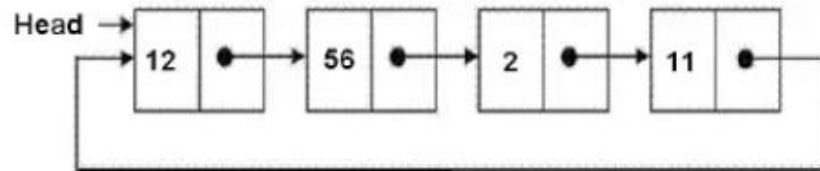
Double Linked List (Dll) merupakan suatu linked list yang memiliki dua variable pointer yaitu pointer yang menunjukan ke node selanjutnya dan pointer yang menunjukan ke node sebelumnya. Setiap head dan tailnya juga menunjukan ke Null. Elemen double linked list terdiri dari 3 bagian yaitu bagian data informasi, pointer next yang menunjukan ke elemen berikutnya dan pointer prev yang menunjukan ke elemen sebelumnya.



Gambar 1.2

c. Circular Linked List

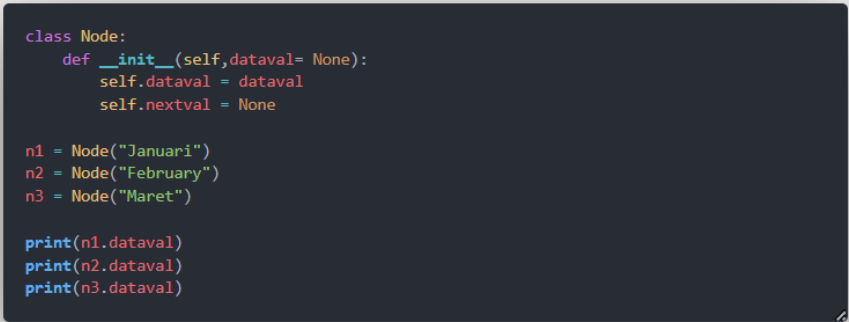
Circular Linked List merupakan suatu linked list dimana tail (node terakhir) menunjukan ke head (node pertama). Jadi tidak ada pointer yang menunjukan NULL.



Gambar 1.3

C. Hasil dan Pembahasan

1. Membuat Linked List



```
class Node:
    def __init__(self, dataval= None):
        self.dataval = dataval
        self.nextval = None

n1 = Node("Januari")
n2 = Node("February")
n3 = Node("Maret")

print(n1.dataval)
print(n2.dataval)
print(n3.dataval)
```

Gambar 1.4

- Langkah awal membuat linked list adalah dengan membuat class dengan nama node
- Selanjutnya dijabarkan dalam fungsi init. Parameter dataval digunakan untuk menyimpan nilai data pada node. Jika tidak ada nilai yang diberikan, defaultnya adalah None
- Self.dataval = dataval ini untuk menetapkan data nilai yang diberikan saat pembuatan elemen kedalam elemen
- Self.nextval = None ini adalah tempat untuk menyimpan referensi ke elemen berikutnya dalam linked list. Awal disetel ke None artinya belum ada elemen berikutnya
- n1, n2, dan n3 adalah Langkah Langkah untuk membuat 3 elemen baru dengan data masing masing Januari, Februari, dan Maret.
- Print(n1.dataval) ini hanya untuk mencetak nilai data dari masing masing elemen

2. Membuat class linked list

```
class Node:
    def __init__(self, dataval= None):
        self.dataval = dataval
        self.nextval = None

class LinkedList:
    def __init__(self, dataval =None):
        self.dataval = dataval
        self.nextval = None

    def listprint(self):
        printval=self.headval
        while printval is not None:
            print (printval.dataval)
            printval = printval.nextval

n1 = Node("Januari")
n2 = Node("February")
n3 = Node("Maret")

Li = LinkedList()
Li.headval = n1

Li.headval.nextval = n2
n2.nextval = n3

Li.listprint()
```

Gambar 1.5

- Untuk class node seperti sebelumnya, kelas ini mendefinisikan sebuah node dalam linked list
- Dataval digunakan untuk menyimpan nilai data pada simpul
- Nextval adalah referensi untuk ke node berikutnya dalam linked list
- Sedangkan kelas linked list ini juga mendefinisikan node
- Def listprint digunakan untuk mencetak nilai data dari setiap node dalam linked list mulai dari node pertama hingga node akhir
- Printval digunakan untuk melacak node saat mencetak
- Tiga objek node dengan nilai januari febuari dan maret
- Untuk objek Li sama dengan linkedlist
- Li.headval diatur ke n1 sehingga menjadi node pertama dalam linked list
- Terakhir listprint dipanggil untuk mencetak nilai dari setiap simpul dalam linked list, yang seharusnya mencetak Januari Febuary dan Maret

3. Penyisipan node diawal pada linked list

```
class Node:
    def __init__(self, dataval= None):
        self.dataval = dataval
        self.nextval = None

n1 = Node("Januari")
n2 = Node("February")
n3 = Node("Maret")

class LinkedList:
    def __init__(self, dataval =None):
        self.headval=None

    def listprint(self):
        printval=self.headval
        while printval is not None:
            print (printval.dataval)
            printval = printval.nextval

    def AddBeginnig(self, newdata):
        NewNode = Node(newdata)
        NewNode.nextval = self.headval
        self.headval = NewNode
```

Gambar 1.6

```
Li = LinkedList()
Li.headval = n1

Li.headval.nextval = n2
n2.nextval = n3

#Menambahkan Awalan Start
print("Menambah awalan start")
Li.AddBeginnig("Start")
```

Gambar 1.7

- Untuk fungsi listprint Sudha dijelaskan diatas selanjutnya adalah fungsi AddBeginning
- Add beginning dengan parameter newdata untuk menambahkan node diawal linked list
- NewNode = Node (newdata) membuat objek baru dari kelasNode dengan nilai data newdata, shingga NewNode menjadi node baru yang akan ditambahkan
- NewNode.nextval = self.headval menetapkan referensi nextval dari node baru (NewNode) ke node pertama saat ini dlam linked list (self.headval)
- Self.headval = NewNode menetapkan headval dari linked list ke node baru (NewNode) Dengan ini, node baru menjadi node pertama dalam linked list
- Dan jangan lupa pada bagian bawah panggil Li.AddBeginnig("start"), maka pada output akan keluar start pada node pertama

4. Penyisipan node di tengah pada linked list

```
class Node:
    def __init__(self, dataval= None):
        self.dataval = dataval
        self.nextval = None

n1 = Node("Januari")
n2 = Node("February")
n3 = Node("Maret")

class LinkedList:
    def __init__(self, dataval =None):
        self.headval=None

    def listprint(self):
        printval=self.headval
        while printval is not None:
            print (printval.dataval)
            printval = printval.nextval

    def AddBeginnig(self, newdata):
        NewNode = Node(newdata)
        NewNode.nextval = self.headval
        self.headval = NewNode

    def AddInbetween(self, mid_node, newdata):
        if mid_node is None:
            print("The mentioned node is absent")
            return
        new_node = Node(newdata)
        new_node.nextval = mid_node.nextval
        mid_node.nextval = new_node
```

Gambar 1.8

```
#Menambahkan middle diantar n1 n2 dan n3
print("Menambahkan middle diantara n1 n2 dan n3")
Li.AddInbetween(n1, "Middle")
Li.AddInbetween(n2, "Middle")
Li.AddInbetween(n3, "Middle")
```

Gambar 1.9

- Selanjutnya adalah AddInbetween dengan parameter mid_node dan newdata
- Pada baris if memeriksa apakah node yang ditentukan (mid_node) ada atau tidak. Jika tidak, cetak pesan dan kembalilah
- new node = Node (Newdata) membuat objek baru dari kelas Node dengan nilai data newdata, sehingga new node menjadi node baru yang akan ditambahkan
- new node.nextval = mid_node.nextval menetapkan referensi nextval dari simpul baru (new node) ke node yang berada setelah simpul yang ditentukan
- untuk baris mid_node.nextval = new_node menetapkan referensi nextval dari simpul yang ditentukan (mid_node) ke simpul baru (new_node). Dengan ini, simpul baru disisipkan di antara node sebelumnya
- Dan jangan lupa panggil Li.AddInbetween(n1,"middle"), akan keluar output middle di tengah n1 n2 dan n3

5. Penyisipan node di akhir pada linked list

```
def AddInEnd(self,newdata):  
    new_node = Node(newdata)  
    if self.headval is None:  
        self.headval = new_node  
        return  
    last = self.headval  
    while(last.nextval):  
        last = last.nextval  
    last.nextval = new_node
```

Gambar1.10

```
print("Menambahkan 'The last' pada akhir data ")  
Li.AddInEnd("The last")
```

Gambar 1.11

- AddInEnd mendefinisikan dengan parameter newdata nilai data untuk simpul baru yang akan ditambahkan di akhir linked list
- new_node membuat objek baru dari kelas Node dengan nilai data newdata, sehingga new_node menjadi node baru yang akan ditambahkan
- baris if untuk memeriksa apakah linked list kosong. Jika iya maka headval dari linked list diatur menjadi node baru, dan metode dihentikan
- baris last untuk menginisialisasi variabel last dengan head dari linked list. Last digunakan untuk melakukan iterasi melalui linked list hingga mencapai node terakhir
- baris while untuk melakukan iterasi selama ada node berikutnya dalam linked list
- baris last memindahkan node berikutnya ke dalam linked list
- untuk baris last,nextval menetapkan dari node terakhir ke node baru. Dengan ini, simpul baru ditambahkan di akhir linked list
- Dan jangan lupa panggil Li.AddInEnd maka outputnya akan keluar The last pada akhir data

6. Menghapus node dengan kata kunci pada linked list

```
def RemoveNode(self, Removekey):  
    Headval = self.headval  
    if Headval.dataval == Removekey:  
        self.headval = Headval.next  
        Headval = None  
        return  
    while Headval is not None :  
        if Headval.dataval == Removekey:  
            break  
  
        prev = Headval  
        Headval = Headval.nextval  
    if Headval == None:  
        return  
    prev.nextval = Headval.nextval  
    Headval = None
```

Gambar 1.12

```
print("Menghapus node dengan kata kunci 'Maret'")  
Li.RemoveNode("Maret")
```

Gambar 1.13

- RemoveNode dengan parameter Removekey atau nilai yang ingin dihapus
- Baris headval digunakan untuk melakukan iterasi melalui linked list
- Baris if untuk memeriksa apakah node pertama memiliki nilai data yang sama dengan Removekey. Jika iya maka node pertama akan dihapus dengan mengatur headval menjadi node berikutnya dan node pertama yang tadi dihapus dari memori
- Baris while melakukan iterasi selama masih ada simpul dalam linked list
- Baris if yang didalam while memeriksa apakah nilai data dari simpul saat ini sama dengan Removekey
- Baris break jika nilai data ditemukan, maka akan keluar dari looping
- Pada baris prev menyimpan simpul saat ini sebagai prev, sehingga nantinya dapat mengatur referensi nextval dari node sebelumnya
- Baris headval pindah ke node berikutnya dalam linked list
- Baris if paling bawah untuk mengatur jika sampai akhir linked list dan nilai data tidak ditemukan, kembalikan
- Baris prev untuk mengatur nextval dari node sebelumnya ke node setelahnya, sehingga node saat ini dihapus dari linked list
- Baris headval untuk menghapus node saat ini dari memori
- Dan jangan lupa panggil Li.RemoveNode("Maret") maka kata kunci yang sama dengan Maret akan dihapus dari memori

7. Menghapus node diawal pada linked list

```
def removeFirst(self):  
    afterhead=self.headval  
    self.headval = afterhead.nextval
```

Gambar 1.14

```
print("Menghapus node pada awal data")  
Li.removeFirst()
```

Gambar 1.15

- RemoveFirst yang bertujuan untuk menghapus node pertama dari linked list
- Baris afterhead menyimpan node yang berada setelah head dalam variable afterhead
- Baris self.head mengatur head dari linked list ke node setelahnya. Dengan ini, simpul pertama dihapus dari linked list
- Dan jangan lupa panggil Li.removeFirst maka node pertama akan dihapus dari memori

8. Menghapus node diakhir pada linked list

```
def RemoveEnd(self):  
    last = self.headval  
    while last is not None:  
        if last.nextval == None:  
            break  
        prev = last  
        last = last.nextval  
    if last == None:  
        return  
    prev.nextval = last.nextval  
    last = None
```

Gambar 1.16

```
print("Menghapus node pada akhir data")  
Li.RemoveEnd()
```

Gambar 1.17

- RemoveEnd yang bertujuan untuk menghapus node terakhir dari linked list
- Baris last untuk menginisialisasi variable last dengan head dari linked list. Last digunakan untuk melakukan iterasi melalui linked list hingga mencapai node terakhir
- Baris while untuk melakukan iterasi selama masih ada node dalam linked list
- Baris if untuk memeriksa apakah simpul saat ini last adalah node terakhir dengan memeriksa apakah node berikutnya adalah None. Jika iya, keluar dari looping
- Baris prev untuk menyimpan simpul saat ini sebagai prev, sehingga nantinya dapat mengatur referensi nextval dari node sebelumnya
- Baris last untuk memindahkan node berikutnya ke linkedlist
- Baris if digunakan untuk jika sampai akhir linked list dan node terakhir tidak ditemukan, kembalikan ke awal
- Baris prev untuk mengatur nextval dari node sebelumnya ke None, sehingga node

terakhir dihapus dari linked list

- Baris last untuk menghapus node terakhir dari memori
- Dan jangan lupa panggil `Li.RemoveEnd()`

D. Latihan

1. Membuat Double linked list degan 3 node didalamnya,kemudian lakukan penyisipan tengah dan hapus dengan kata kunci!

```
# Membuat class Node
class Node:
    def __init__(self, data):
        self.data = data
        self.nextData = None
        self.prevData = None

# Membuat node
n1 = Node("Noufal")
n2 = Node("Aji")
n3 = Node("Prasetyo")

# Class Double Linked List
class DoublyLinkedList:
    def __init__(self):
        self.head = None

# Print Double Linked List
def listprint(self):
    printval = self.head
    while printval is not None:
        print(printval.data)
        printval = printval.nextData
    print("\n")

# Menyisipkan data
def AddInbetween(self, mid_node, newdata):
    if mid_node is None:
        print("Node yang disebutkan tidak ada")
        return
    new_node = Node(newdata)
    new_node.nextData = mid_node.nextData
    if mid_node.nextData:
        mid_node.nextData.prevData = new_node
    mid_node.nextData = new_node
    new_node.prevData = mid_node

# Hapus dengan kata kunci
def RemoveNode(self, Removekey):
    current = self.head
    if current is not None:
        if current.data == Removekey:
            self.head = current.nextData
            if current.nextData:
                current.nextData.prevData = None
            current = None
            return

    while current is not None:
        if current.data == Removekey:
            break
        prev = current
        current = current.nextData

    if current is None:
        return

    prev.nextData = current.nextData
    if current.nextData:
        current.nextData.prevData = prev
    current = None

# Membuat objek DoublyLinkedList
dll = DoublyLinkedList()

# Mengatur head dari DoublyLinkedList
dll.head = n1

# Mengatur node berikutnya
n1.nextData = n2
n2.prevData = n1
n2.nextData = n3
n3.prevData = n2

# Tugas
print("Membuat Double linked list ")

# Menampilkan list data awal
print("Memamipkan list data Double linked list :")
dll.listprint()

# Menambahkan data di antara n1 dan n2
dll.AddInbetween(n1, "Aji")
print("Setelah dilakukan penyisipan data di tengah 'Aji' dengan Double linked list :")
dll.listprint()

# Menghapus node dengan kunci 123
dll.RemoveNode("Aji")
print("Setelah dilakukan remove dengan kunci 'Aji' dengan Double linked list :")
dll.listprint()
```

Gambar 1.18

- Membuat kelas node yang memiliki attribute data untuk menyimpan nilai data,nextData untuk ke node berikutnya dan prevData untuk ke node sebelumnya
- Membuat 3 node dengan nilai Noufal Aji Prasetyo
- Mendefinisikan kelas Doublelinkedlist yang memiliki attribute head untuk menyimpan ke node pertama dari linked list
- Listprint untuk mencetak nilai data dari setiap node dalam linked list,dimulai dari head hingga node terakhir
- AddInBetween untuk menyisipkan node baru diantara node yang sudah ada dalam linked list
- RemoveNode untuk menghapus node dari linked list berdasarkan nilai data yang sama dengan kata kunci yang mau dihapus
- Dan jangan lupa untuk memanggil setiap fungsi agar fungsi nya berjalan

```
Membuat Double linked list
Memampikan list data Double linked list :
Noufal
Aji
Prasetyo

Setelah dilakukan penyisipan data di tengah 'Aji' dengan Double linked list :
Noufal
Aji
Aji
Prasetyo

Setelah dilakukan remove dengan kunci 'Aji' dengan Double linked list :
Noufal
Aji
Prasetyo
```

Gambar 1.19

- Gambar diatas adalah hasil dari operasi atau code dari gambar 1.15

2. Membuat circular linked list dengan 3 node di dalamnya, kemudian lakukan penyisipan akhir dan hapus akhir!

```
# Membuat class Node
class Node:
    def __init__(self, data):
        self.data = data
        self.nextData = None

# Class Circular Linked List
class CircularLinkedList:
    def __init__(self):
        self.head = None

# Menambahkan fungsi appen pada class circular linked list
def append(self, data):
    newNode = Node(data)
    if not self.head:
        self.head = newNode
        newNode.nextData = self.head
    else:
        temp = self.head
        while temp.nextData != self.head:
            temp = temp.nextData
        temp.nextData = newNode
        newNode.nextData = self.head

# menambahkan fungsi list
def listprint(self):
    if not self.head:
        return
    temp = self.head
    while True:
        print(temp.data, end=" ")
        temp = temp.nextData
        if temp == self.head:
            break

# Menyisipkan data pada akhir
def addInEnd(self, newData):
    newNode = Node(newData)
    if self.head is None:
        self.head = newNode
        return

    last = self.head
    while(last.nextData and last.nextData != self.head):
        last = last.nextData
    last.nextData = newNode
    newNode.nextData = self.head

def removeEnd(self):
    last = self.head
    prev = None
    while last.nextData and last.nextData != self.head:
        prev = last
        last = last.nextData
    if last is None:
        return
    if prev is None:
        self.head = None
    else:
        prev.nextData = last.nextData
    last = None

# Membuat objek CircularLinkedList
c11 = CircularLinkedList()

# Tugas
print("Membuat circular linked list")
print("Menampilkan list circular linkedlist")
c11.listprint()

# Menambahkan data ke circular linked list
c11.append("Noufal")
c11.append("Aji")
c11.append("Prasetyo")
c11.listprint()
print("\n")

#menyisipkan data akhir
c11.addInEnd("Ganteng")
print("Menyisipkan data terakhir 'Ganteng' menggunakan circular linked list")
c11.listprint()
print("\n")

#Hapus data Akhir
c11.removeEnd()
print("Setelah dilakukan penghapusan data akhir 'Ganteng' menggunakan circular linked list")

# Menampilkan circular linked list
c11.listprint()
```

Gambar 1.20

- Class node yang memiliki attribute data untuk menyimpan nilai data dan nextData untuk ke node berikutnya

- CircularLinkedList yang memiliki attribute head untuk menyimpan ke node pertama dari linked list
- Menambahkan fungsi appen untuk menambahkan node ke circular linked list. Jika linked list masih kosong, node baru menjadi head dan mengarah ke dirinya sendiri. Jika tidak simpul baru ditambahkan ke di akhir dan dihubung kembali ke head
- Menambahkan fungsi listprint untuk mencetak data dari setiap node dalam circular linked list, diulai dari head hingga kembali ke head
- addInEnd untuk menyisipkan node baru diakhir Circular linked list
- removeEnd untuk menghapus node terakhir dari circular linked list
- Dan jangan lupa panggil fungsinya agar berfungsi tidak eror

```
Membuat circular linked list
Menampilkan list circular linkedlist
Noufal Aji Prasetyo

Menyisipkan data terakhir 'Ganteng' menggunakan circular linked list
Noufal Aji Prasetyo Ganteng

Setelah dilakukan penghapusan data akhir 'Ganteng' menggunakan circular linked list
Noufal Aji Prasetyo
```

Gambar 1.21

- Gambar diatas adalah hasil dari kode pada gambar 1.20

E. Kesimpulan

Kesimpulan dari praktikum di atas adalah mengetahui konsep konsep dasar terakit struktur data linked list dan variasinya seperti single linked list daouble linked list,dan circular linked list.Dengan memahami konsep dan implementasi dari berbagai jenis linked list, kita dapat memilih struktur data yang paling sesuai untuk kebutuhan spesifik dalm pengembangan perangkat lunak